Retrieval Augmented Generation Fine-Tuned LLM Model for Code Recommendations to Mitigate Lock Contention

Ashadullah Shawon
ashadullah.shawon@ontariotechu.net
Ontario Tech University
Electrical, Computer, & Software Engineering
Oshawa, Ontario , Canada

Ramiro Liscano
ramiro.liscano@ontariotechu.ca
Ontario Tech University
Electrical, Computer, & Software Engineering
Oshawa, Canada

Akramul Azim
akramul.azim@ontariotechu.ca
Ontario Tech University
Electrical, Computer, & Software Engineering
Oshawa, Ontario , Canada

Vijay Sundaresan
vijaysun@ca.ibm.com
IBM Toronto Lab
Toronto, Ontario , Canada

Yee-Kang Chang
yeekangc@ca.ibm.com
IBM Toronto Lab
Toronto, Ontario , Canada

Abstract

Lock contention performance faults can lead to degradation in the performance of software applications. Unlike software bugs, performance faults do not lead to failures and application crashes but surface as a degradation in the response and execution of an application and can surface fairly late in the deployment life of an application. Tools exist for the identification and detection of lock performance faults but there is a lack of effective code refactoring recommendations for a developer to mitigate the performance degradation caused by lock-contention. Recent advances in Large Language Models (LLMs) have demonstrated positive results in code refactoring for fixing software bugs and mitigating run time faults. However, traditional LLM-based approaches often suffer from hallucination errors, where the generated code may not accurately reflect the context of the project or existing codebase. This thesis presents a novel approach that combines Retrieval Augmented Generation (RAG) with a fine-tuned LLM model for refactored code recommendation aimed at reducing lock-contention performance faults in Java applications. The RAG fine-tuned model combines the strengths of contextual understanding from LLMs with the precision of retrieval-based systems, thereby ensuring that the generated recommendations are relevant, accurate, and hallucination-free. Semantic and syntactic metrics of the recommendations generated by the combined RAG and LLM model show an accuracy of approximately 90% compared to an accuracy of approximately 25% when a baseline LLM model is used.

CCS Concepts

• **Software and its engineering** → *Maintaining software*;

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICPE Companion '25, Toronto, ON, Canada.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1130-5/25/05
https://doi.org/10.1145/3680256.3721324

Keywords

Code Smell, Refactored Code Recommendation, Lock Contention, Large Language Model, RAG, Fine-Tuning

ACM Reference Format:
Ashadullah Shawon, Ramiro Liscano, Akramul Azim, Vijay Sundaresan, and Yee-Kang Chang. 2025. Retrieval Augmented Generation Fine-Tuned LLM Model for Code Recommendations to Mitigate Lock Contention. In *Companion of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE Companion '25), May 5–9, 2025, Toronto, ON, Canada.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3680256.3721324

1 Introduction

In multi-threaded applications, lock contention is a significant performance bottleneck that can reduce the overall efficiency and scalability of a software application. These lock contention faults can have far-reaching implications, as seemingly negligible delays of milliseconds can potential balloon into substantial financial losses.

Even though lock contention faults have been extensively studied [3] and performance analysis tools such as JProfiler, VisualVM, and JLM [12] exist to identify and locate these lock contention faults these tools are manually intensive and require considerable expertise in the tool itself and do not offer recommendations to reduce the lock contention. The manual identification and refactoring of these lock contention faults are time-consuming and prone to human oversight. This underscores the need for automated tools that can efficiently recommend improvements in the code [10].

This paper presents a groundbreaking approach in generating code recommendations to mitigate lock contention faults by integrating Retrieval Augmented Generation (RAG) [14] with a Fine-Tuned Large Language Model (LLM) based on lock contention code smells. By integrating RAG's contextual retrieval capabilities with the LLM's understanding, our approach enables context-aware code analysis with personalized code recommendations streamlined for the detection of lock contention performance faults.

Recent advancements in Large Language Models (LLMs) [16] have shown promise in assisting with software engineering tasks [13], including code smell and bug detection [7][5]. However, LLMs face limitations such as hallucinations, output inconsistency, difficulty leveraging contextual knowledge, and inability to perform dynamic analysis [8]. One of the most significant drawbacks of

these models is their propensity for hallucinations, a phenomenon where they generate text that is not grounded in reality, but rather a product of their own imagination. This can lead to the creation of factually incorrect, misleading, or even harmful content, which can have serious consequences in real-world applications.

It is important to note that the approach presented in this paper is a static code analysis approach which would take place after the lock contention is identified and located, as not all lock contention code smells would lead to lock contention as the run time conditions may not trigger the contention.

Once the contentious lock and its location are identified the code with the suspected lock can be extracted and processed by the code analysis approach described in this paper in order to generate a recommended change to the code.

In summary, our contributions are listed below:

(1) We created a lock contention fault code-smell dataset for fine-tuning LLMs for Java code.
(2) We fine-tuned the baseline LLAMA3.1-8B model and integrated it into our RAG pipeline to build a better code recommendation system for Java lock contention performance faults.

2 Related Works

Despite significant research on identifying lock contention faults in multi-threaded applications [1] [2] [10], existing approaches often do not provide concrete refactored code recommendations to mitigate lock-contention related performance issues. Many studies focused on specific methods for detecting lock contention but failed to offer practical recommendations for improving the underlying code. As a result, developers are left without actionable guidance to address the code smells from lock contention issues in their source code.

In a similar study, Hofer et al. [10] developed an approach to identify lock contention in Java applications by tracking locking events through the JVM. Their method involves tracing the call chains of threads involved in contention, allowing for precise detection of lock contention issues. However, this approach is focused on identifying and locating the locks and does not present any solutions to reduce the lock contention.

Similarly, Alam et al. [3] conducted a systematic categorization of performance bugs and corresponding solutions. As part of this effort, they designed and developed a specialized tool called SyncPerf which is capable of identifying substantial synchronization-related performance defects and guiding developers in diagnosing the root causes of lock contention issues. Notably, SyncPerf provides suggestions in the form of concise, human-readable English descriptions, although it does not generate accompanying refactored code examples.

Mashhadi et al. [17] introduced an innovative, automated program repair framework by fine-tuning CodeBERT to identify straightforward Java bugs directly from source code. Although this system represents a promising approach to automated bug fixing, it's recommendation system was not able to provide accurate results as BERT model is not able to generate high quality recommendations due to it's structure.

A growing trend among researchers involves improving the capabilities of Large Language Models (LLMs) to address code quality issues. Silva et al. [22] fine-tuned the LLAMA2 baseline model and created a new recommendation model called RepairLLAMA for program repair to reduce the vulnerabilities in source code but they didn't integrate RAG with their fine-tuned model to reduce the hallucination issues.

Recently, Shawon et al. [21] fine-tuned the baseline LLAMA2 model based on lock contention dataset. The fine-tuned model called FaultLLAMA2 can generate both general and refactored code recommendations to reduce the lock contention issues from the given source code. However, the FaultLLAMA2 model also generates hallucinations and inconsistent recommendations.

3 Methodology

Our proposed method consists of two main sections: the RAG Pipeline and the fine tuned LLM. These two components work together to provide a robust and accurate recommendation system tuned for Java lock contention faults. Our recommendation system takes as input an example of Java source code that is causing the excessive lock contention and a prompt requesting an alternate code to reduce the lock contention. The system then uses the RAG pipeline and fine-tuned LLM to generate refactored code recommendations that reduce the lock contention. The system provides a general recommendation as text as well as refactored code to help developers improve the performance of their code. An example of this prompt and the input code and output of the systems is shown in Listing 1 which is taken from a Java open source project.

Figure 1 shows the major components of our architecture including the flow of data through the architecture. A detail description of these major components follows commencing with the fine tuned LLM component.

3.1 Fine Tuned LLM

For this research, we utilized the smallest LLAMA3-8B-instruct model as our baseline due to constraints in computing power. To adapt this model to specific knowledge, we fine-tuned it by training it on set of Java lock contention code smells that contains the Java lock contention code snippets with detailed as well as general and refactored code recommendations to reduce the lock contention [2] [1].

Fine-tuning an LLM involves customizing the model's neural network weights to generate relevant outputs or recommendations for the target task [23]. We applied a two-step fine-tuning process. First, we identified the most efficient model parameters leveraging the LoRA (Low-Rank Adaptation) [11] Parameter-Efficient Fine-Tuning (PEFT) approach [9], and then we applied supervised fine-tuning to adjust the model's learned representations and produce accurate outputs.

We applied Parameter-Efficient Fine-Tuning (PEFT) which is a one kind of fine-tuning process that fine-tuned the baseline LLAMA3.1-8b-instruct model without updating all the weights [9]. To fine-tune the model, we updated a small subset of the baseline model's parameters using the LoRA configuration. We set up the LoRA configuration by specifying key parameters such as *lora alpha*, *lora dropout*, *rank (r)* , *bias*, and *task_type*. We employed a random

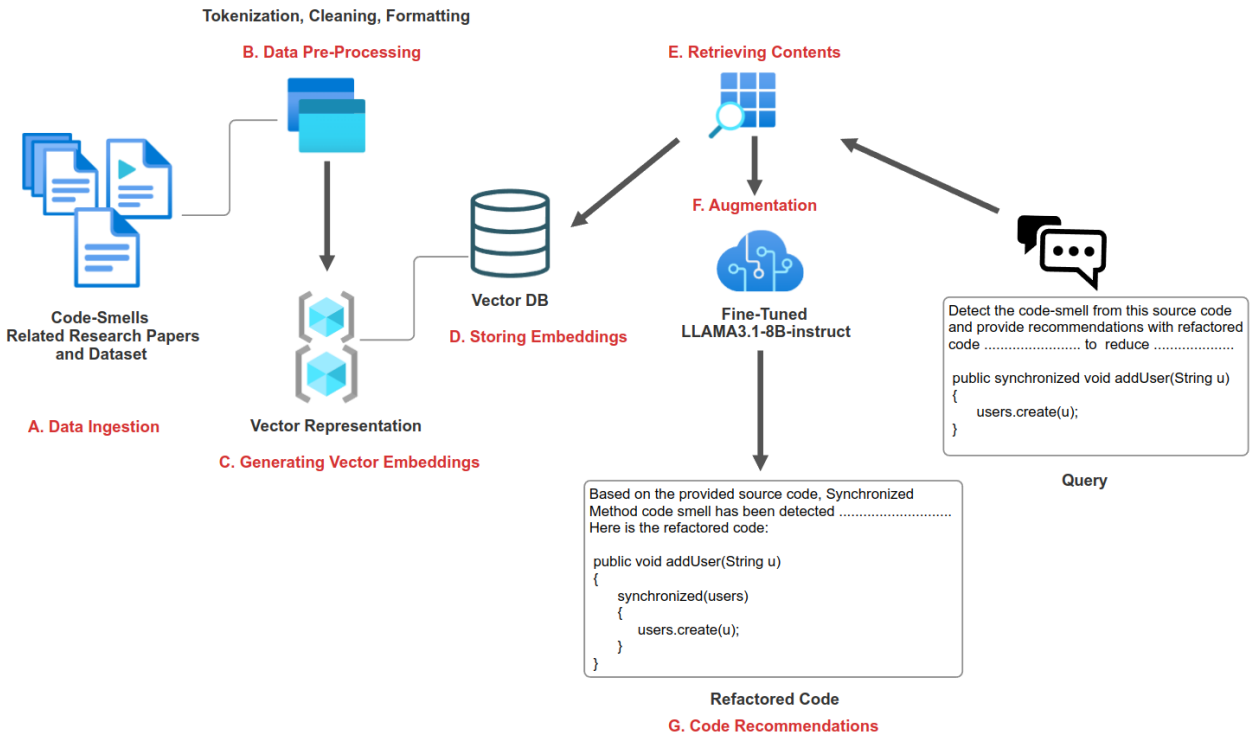

Figure 1: Proposed RAG powered code recommendation system

search algorithm to find the optimal values for these parameters, which are crucial in determining the number of trainable parameters. The rank (r) of the adapter affects the inner dimension of the low-rank matrices with higher ranks resulting in more trainable parameters. The specific values used for the LoRA configuration are listed in Table 1.

Table 1: LoRA Configuration Value

Parameters	Values
lora_alpha	16
lora_dropout	0.1
r	64
bias	none
task_type	CAUSAL_LM

We employed a process called Supervised Fine-Tuning (SFT) [15] to adapt the model to our specific task of reducing code smells which triggers the lock contention issues from given source code so that the fine-tuned model can generate refactored code recommendations. We trained the model on the labeled dataset tailored to our domain of interest consisting of input-output pairs where each input (e.g., a Java source code with a prompt) corresponds to a specific output (e.g., a general recommendation with refactored code). This process enabled the model to grasp domain-specific knowledge and generate more precise responses, thereby increasing its utility.

Our fine tuning dataset is structured to facilitate the refactoring of code smells that triggers the lock contention issues. Tables 2 and Table 3 shows the detailed columns of our lock contention dataset. We created the dataset according to the fine tuning dataset instructions [24] from the LLAMA3. The first 2 columns are shown in Table 2 and last 2 columns are shown in Table 3.

The first column contains a prompt or instruction that guides the model in identifying the code-smell and providing recommendations to reduce or fix it. The second column holds an example of a Java source code with code smells that triggers the lock contention issue. The third column provides the expected output which includes a description of the code-smell and a refactored source code recommendation that resolves the issue. The fourth column combines the prompt and the values from the first three columns.

Table 2: Sample Instruction and Input columns of our dataset

Instruction	Input

Detect the code-smell from this source code and provide recommendations with refactored code to reduce that code-smell which triggers the lock-contention issues	`public synchronized void addUser(String u)`
	`{`
	`users.create(u);`
	`}`

```
User Prompt:
Detect the code-smell from this source code and provide
↪  recommendations with refactored code to reduce that code-smell
↪  which triggers the lock-contention issues.

public class Grocery {
    private final ArrayList fruits = new ArrayList();
    private final ArrayList vegetables = new ArrayList();
    public synchronized void addFruit(int index, String fruit) {
        fruits.add(index, fruit);
        }
    }

Result:
Based on the provided source code, Synchronized Method code smell hs
↪  been detected that can trigger lock-contention issues:

Recommendations with refactored code:
Instead of synchronizing the entire method, synchronize only the
↪  critical section of the code that accesses shared resources.

public class Grocery {
    private final ArrayList fruits = new ArrayList();
    private final ArrayList vegetables = new ArrayList();
    public void addFruit(int index, String fruit) {
        synchronized (fruits) { // Synchronize only the fruits list
            fruits.add(index, fruit);
        }
    }
}

By synchronizing only the critical section of code that accesses
↪  shared resources, we can reduce lock-contention issues and
↪  improve performance.
```

Listing 1: Code Recommendations from RAG+Fine-Tuned LLM Model

To prevent overfitting on the specialized dataset, we adjusted key training parameters such as the learning rate, batch size, and training epochs. We used a random search algorithm [4] to find the optimal supervised training parameters, ensuring that the model performed well on both our specific task and broader tasks. The values of these parameters are listed in Table 4.

3.2 The RAG Pipeline

The Retrieval Augmented Generation (RAG) system enhances the capabilities of Large Language Models (LLMs) by incorporating real-time, relevant information into their outputs. The RAG pipeline is a dynamic workflow that retrieves, ranks, synthesizes, and generates responses based on the most up-to-date information. Thus, the RAG system ensures that the output of the LLM is grounded in relevant and accurate information which ensures more informative and accurate responses. The Retrieval Augmented Generation (RAG) process is composed of a series of critical steps that work together to generate accurate and informative responses. These key steps are described below.

Data Ingestion (Step A in Figure 1): We initiated the proposed research methodology by collecting relevant, domain-specific textual data from various external sources such as existing code-smells related research papers and code-smells dataset. These documents and the dataset represent a wealth of raw data that serves as the foundation for building a tailored knowledge base which our system queries during the retrieval process to enhance its ability to respond effectively and provide accurate recommendations for mitigating lock contention code-smells.

At first, we parsed the full contents of the research papers [1] [2] and the dataset using the PDF loader from LLAMAIndex framework. Then we converted the contents into text files. These text files are the raw contents of our proposed method.

Table 3: Sample Output and text columns of our dataset

Output	text

Based on the provided source code, Synchronized Method code smell has been detected that can trigger lock-contention issues. Instead of guarding the entire addUser method, we can guard the users object with a separate lock. Here's the refactored code:
`public void addUser (String u)`
`{`
`  synchronized(users){`
`  users.create(u);`
`  }`
`}`
	System Prompt: Below is an instruction that describes a task. Write a response that appropriately completes the request. ### Instruction: Detect the code-smell from this source code and provide recommendations with refactored code to reduce that code-smell which triggers the lock-contention issues
	`public synchronized void addUser(String u)`
	`{`
	`  users.create(u);`
	`}`
	### Response: Based on the provided source code, Synchronized Method code smell has been detected that can trigger lock-contention issues. Instead of guarding the entire addUser method, we can guard the users object with a separate lock. Here's the refactored code:
	`public void addUser (String u)`
	`{`
	`  synchronized(users){`
	`  users.create(u);`
	`  }`
	`}`

Table 4: Optimal SFT Parameters

Parameters	Values
num_train_epochs	1
train_batch_size	4
learning_rate	2e-4
optimizer	AdamW
weight_decay	0.001

Data Pre-Processing (Step B in Figure 1): We started pre-processing the data once the documents were loaded, parsed, and converted to text files. One common transformation technique is text-splitting which breaks down lengthy text into smaller and more digestible segments. Furthermore, we preprocessed the data to create smaller and indexable units which is knows as tokens. Tokenization is the most important part of data pre-processing as the quality of data is dependent on important tokens. This pre-processing step is crucial to ensure that the retrieval process is both accurate and efficient and this process allows us to effectively query and retrieve relevant information from the dataset. Listing 2 shows the tokenization steps of data pre-processing.

Listing 2: Data Tokenization

```
[Synchronized, method, refers, to,
synchronization, mechanism, that,
protects, critical, regions,.....,....,public,
synchronized,void,addUser,
String,u,users,create,u]
```

Generating Vector Embeddings (Step C in Figure 1): We enabled the efficient processing of ingested data that can be transformed into a suitable format. This is achieved by generating high-dimensional vector embeddings which convert text into a numerical format that can be easily processed by the system. We transformed the data chunks into vector representations using advanced embedding models. These vector embeddings capture the essence of the text which allow us to perform accurate similarity searches. Then, we stored the resulting vector representations in a Vector Store which is a specialized database designed for rapid retrieval based on similarity metrics, enabled us to quickly identify relevant information.

Listing 3: Vector Embeddings

```
[[0.2678,0.3278,....,0.1562],
.............................
[0.7129,0.6028,....,0.2921]]
```

Storing Embeddings (Step D in Figure 1): We stored the preprocessed data and generated embeddings in specialized vector databases which are optimized to efficiently handle high-dimensional vector data. We utilized a vector database called *ChromaDB* to facilitate fast search and retrieval operations and it ensures that the stored information remains readily accessible and can be quickly retrieved in real-time. We can ensure that our system can rapidly retrieve relevant information if we store data in this way and it also enables the swift and accurate responses to user queries.

Retrieving Context Relevant Contents (Step E in Figure 1): We utilized the retrieving controller of the RAG pipeline to retrieve the contents from the vector database. When a user inputs a query into our system then it is transformed into a vector embedding and matches the format of the documents stored in the vector database. The query is used to search the vector store where the most relevant and up-to-date information related to the query is identified and retrieved. The system is able to efficiently retrieve the most pertinent information by transforming the query into a vector embedding. It also ensures that responses are accurate and aligned to context.

In this research, we utilized both vector retriever and BM25 retriever which is a hybrid retriever approach to search for lock contention code-smells related documents using the BM25 algorithm [20] and re-ranked the retrieved contents using similarity score. We applied the B25 retriever to retrieve the top 25 responses from the embeddings in vector DB based on the occurrence and rarity of query terms across the corpus which is called similarity score. Thus it's more accurate and efficient search process with ranked documents and it's more relevant to the query.

Table 5 shows an output of this step which contains the top 25 contents along with the similarity score to the prompt.

Table 5: Similarity score of Top 25 Contents

Contents	Similarity Score
1.Synchronized Method refer.....	0.95
2.Significant slowdowns due to long-held	0.95
3.Synchronize on specific objects.....	0.94
.....	....
10. public class SynchronizedMethod...	0.81
.....	....
25. synchronized(this)...	0.77

Augmentation (Step F in Figure 1): In this step, we augmented the retrieved data from step E and the user prompt with the fine-tuned LLAMA3.1-8b-instruct model. Our proposed fine-tuned model is integrated with the RAG pipeline in this step. This process also augments the original prompt using fine-tuned LLM with the new retrieved information and relevant context from step E. The retrieved information actively influences the model's response. This ensures that the response remains contextually relevant rather than relying purely on LLAMA's pre-trained knowledge. The augmentation helps the fine-tuned LLM to remove the hallucination or wrong recommendation by checking or verifying from the external RAG retrieved contents. The augmented prompt is shown below and algorithm 1 shows the algorithms of retrieve, augmentation, and generation steps.

Augmented Prompt: Analyze the given Java source code and identify any code smells related to lock contention. Use the following lock contention code smell categories: Synchronized Method, Unified Locking, Same Lock, Loop Inside Critical Section (LIC), Loop Outside Critical Section (LOC), and Overly Split Locks. For each identified code smell, provide an explanation of why it causes contention issues and suggest refactored code to mitigate the problem. The recommendations should follow best practices such as lock splitting, re-entrant locks, atomic types, and parallelization where applicable. If multiple code smells are present, prioritize solutions that reduce lock contention and improve performance. Format the response as a structured analysis with detected code smells, explanations, and refactored code snippets..

Input: Large Language Model LLM, knowledge base KB, user query Q
Output: Augmented response R
Function `RetrieveRelevantDocuments`(KB, Q):
  Search KB for relevant documents matching Q;
  Rank and filter the most relevant documents;
  return set of retrieved documents D
Function `Augmentation`(Q, D):
  Integrate RAG with fine-tuned LLAMA3.1-8b-instruct model Concatenate D with Q to form augmented prompt Q_A;
  Format Q_A to ensure context relevance;
  return augmented prompt Q_A
Function `GenerateResponseWithLLM`(LLM, Q_A):
  Feed Q_A into LLM for response generation;
  Generate response R considering retrieved context;
  return generated response R
Initialize $D \leftarrow$ `RetrieveRelevantDocuments`(KB, Q);
Initialize $Q_A \leftarrow$ `Augmentation`(Q, D);
Initialize $R \leftarrow$ `GenerateResponseWithLLM`(LLM, Q_A);
return R;

Algorithm 1: Algorithm of retrieve, augmentation, and generation steps

3.3 Prompt Engineering

Our experiments revealed that incorporating specific keywords and phrases related to the expected output yielded the best results. Finally, we selected a prompt that meet the requirements of our expected recommendations. Prompt: **"Detect the code smell from this source code and provide recommendations with refactored code to reduce that code-smell which triggers the lock-contention issues."**

4 Creation of the Java Lock Contention Code Smells Dataset

To ensure the dataset is unbiased, it's recommended to collect data from diverse sources. Our code-smell dataset includes examples of code smells, general recommendations, and refactored code to reduce the code smells. A large dataset is not required for fine-tuning as we didn't train the baseline model from scratch. In fact, previous researchers have shown that fine-tuning a base model with a small dataset can yield promising results where the authors achieved good results with just a few chosen examples [25].

Our research group identified 6 different types of Java synchronized lock contention code smells [1][2] that were used as a basis for the creation of our dataset and to better understand the code smells, we have included a code snippet in listing 4 from the *Synchronized Method* type code-smell that demonstrates how entire regions of code are unnecessarily synchronized, rather than isolating the synchronization to the specific critical region that requires it. The 5 other lock contention code smells are described in more detail in [1].

Our research group also developed a static code analysis tool which can be used to extract the code snippets of the code smells.

```java
public class MonitoringInterface {
    private List<User> users;
    private List<Query> queries;
    public synchronized void addUser(String u) {
         users.add(u);
    }
    public synchronized void removeQuery(String q) {
        queries.remove(q);
    }
}
```

Listing 4: Synchronized Method Code-Smell

The tool also generates the general text-based recommendations. We used that tool to extract the 575 unique code-snippets of the code smells from three different open source projects (*Apache Hbase*, *Glide*, and *EventBus*). Then we collected the refactored code based on the general recommendations from that tool.

Figure 2 is a summary of the Java lock contention code smells dataset collection process previously described.

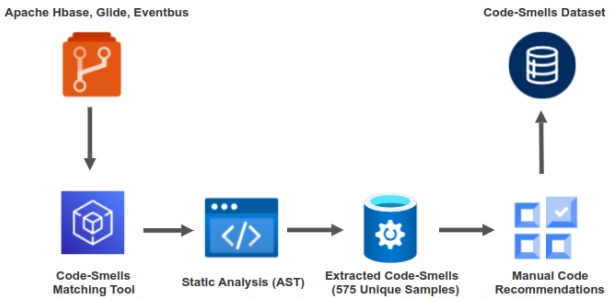

Figure 2: Proposed code smells dataset collection process

5 Experiments and Result Analysis

Our research experiment was conducted on a high-performance computing system equipped with an Intel 16-core processor, a T4 Tesla GPU, and 64GB of RAM. We selected Python 3 as the programming language for our experiment.

We split the dataset into training and testing sets. This allowed our fine-tuning to achieve proper model development, hyperparameter tuning, and evaluation. We used a splitting ratio of 80:20 where 80% data was used for fine-tuning and 20% was used for testing.

5.1 Evaluation Metrics

We evaluated our proposed research works using different kinds of evaluation metrics. As our proposed fine-tuned model recommends both the solution and refactored code, so we need to validate the generated solution, refactored code correctness, syntactic, and semantic matching score. We monitored training loss during fine-tuning and evaluated our test results by BLEU, CodeBLEU, and Pass@K metrics. We evaluated the correctness of the recommended solutions using BLEU metrics [18], the syntactic and semantic accuracy of the refactored code using CodeBLEU metrics [19], and the

Table 6: Experimental Results of RAG+Fine-Tuned LLAMA3.1-8b-instruct Model, Fine-Tuned LLAMA3.1-8B-instruct and Baseline LLAMA3.1-8B-instruct (BLEU, CodeBLEU, Pass@K)

Model	BLEU	CodeBLEU	pass@10	pass@100
RAG+Fine-Tuned LLAMA3.1-8b-instruct	0.92 ± 0.005	0.90 ± 0.004	0.52 ± 0.005	0.80 ± 0.005
Fine-Tuned LLAMA3.1-8B-instruct	0.75 ± 0.005	0.68 ± 0.005	0.31 ± 0.005	0.50 ± 0.005
Baseline LLAMA3.1-8B-instruct	0.31 ± 0.006	0.24 ± 0.005	0.20 ± 0.005	0.31 ± 0.005

functional correctness of the refactored code using Pass@K metrics [6]. Our evaluation framework considered multiple metrics to provide a comprehensive assessment of our model's performance.

In our experiment, the training process consisted of 1 epoch with 900 steps and the training loss value was recorded at each step. A decline in training loss across epochs typically signifies that the model is acquiring knowledge and enhancing it's performance on the training data. Figure 3 illustrates the training loss curve of our proposed fine-tuned model offering a visual representation of it's learning trajectory.

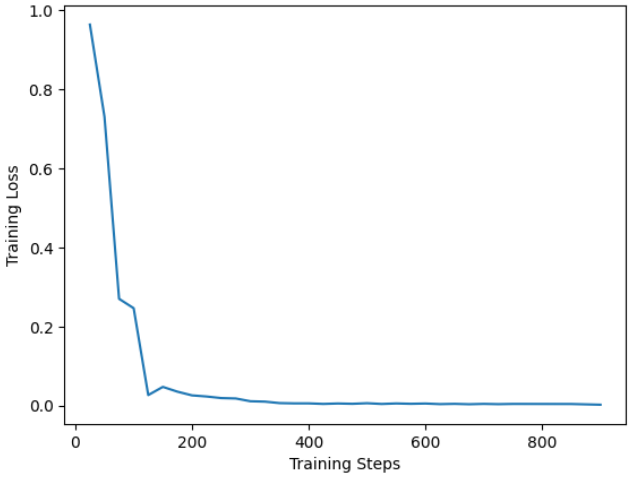

Figure 3: Training Loss Curve of Fine-Tuned LLAMA3.1-8B-instruct Model

5.2 Results

To assess the performance of our RAG powered fine-tuned language model, we evaluated its performance on the test dataset using three metrics: BLEU, CodeBLEU, and Pass@K. These metrics provide a comprehensive evaluation of the model's capabilities assessing the general recommendation while the CodeBLEU and Pass@K evaluate the refactored code recommendations.

We calculated the average BLEU and CodeBLEU scores over 10 independent runs with different samples of test dataset at each time to ensure a reliable measure of our proposed model's performance.

We also compared the results of our RAG+Fine-Tuned LLAMA3.1-8B-instruct model with the Fine-Tuned LLAMA3.1-8B-instruct and the baseline LLAMA3.1-8B-instruct model. We obtained 3 times better results than the baseline model and 1.2 times better results than the fine-tuned model. Table 6 shows the experimental results with a detailed comparison of the performance of the models using the BLEU, CodeBLEU, and Pass@K metrics. Table 6 and Figure 4 shows the experimental results of our research.

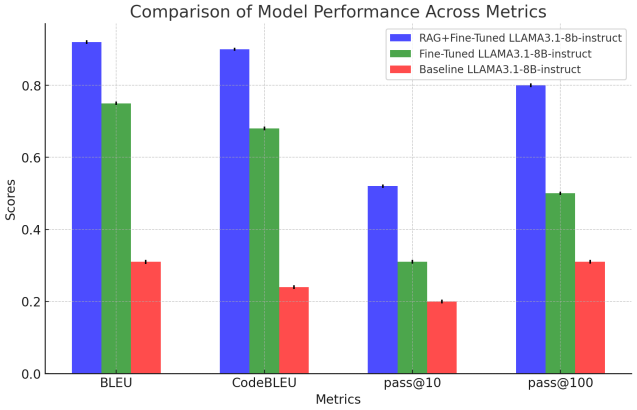

Figure 4: Comparison of model performance across metrics

We have also tested our model by an external open source project called Apache Kafka. We extracted the 6 different types of lock contention prone code smells from the Apache Kafka and asked our proposed model to fix the code smells that are the reasons of performance issue due to lock contention. Table 7 shows the result of this experiment.

Table 7: Effectiveness of Proposed Model in Fixing Lock Contention-Related Code Smells in Apache Kafka

Code Smell Type	Fixed Code / Total Code
Unified Locking	76 / 84
Same Lock	50 / 56
Synchronized Method	120 / 133
Loop Inside	160 / 178
Overly Split	24 / 27
Loop Outside	33 / 38

Our evaluation results demonstrate the effectiveness of our proposed model in generating accurate recommendations and refactored code. Specifically, we achieved an 92% match between our model's general recommendations and the actual correct recommendations, as measured by the BLEU score.

Furthermore, the CodeBLEU score revealed that 90% of the refactored code generated by our model matched the actual refactored code which indicates a high degree of accuracy in code generation.

Notably, our results also showed that increasing the number of attempts or entries (K) in the Pass@K metrics led to better performance in unit testing which highlights the model's ability to improve its accuracy with more tries.

By evaluating the performance of our proposed RAG based fine-tuned model using multiple metrics and comparing it with the baseline and fine-tuned model, we demonstrate the effectiveness of our proposed RAG pipeline with fine-tuned model approach in improving the model's recommendation and code generation capabilities.

Overall, we ensured fair and valid evaluation results. Our evaluation results suggest that our RAG+Fine-Tuned LLAMA3.1-8b-instruct model outperforms the both baseline model and fine-tuned LLAMA3.1-8b-instruct model in generating accurate recommendations and refactored code which demonstrates the effectiveness of our approach in improving the model's performance.

6 Conclusion

In this paper, we presented a RAG fine-tuned model that leverages the strengths of both retrieval-augmented generation and fine-tuning to generate general recommendations as well as refactored code recommendations that mitigates code smells related to lock contention issues. Our approach combines the efficiency of RAG with the adaptability of fine-tuning which enables the model to learn from the external dataset and adapt to new domain-specific task. We demonstrated the effectiveness of our model in generating accurate and relevant code recommendations, outperforming state-of-the-art baselines. However, our proposed model is highly dependent on external RAG contents and can not evaluate the performance of the refactored code real time to verify the output. In this research work, we worked with only one LLM model called LLAMA3 but there are other state-of-the art LLM models and it will be a better approach if we upgrade the existing single model recommendation system to multi LLM models recommendation system as we can mitigate bias and improve overall quality of the recommendations. Furthermore, we didn't find any large lock contention code-smells benchmarks to evaluate our model. In future, we will address the mentioned limitations and explore new approaches such as integrating agent based RAG pipeline with our model to enhance the accuracy and reliability of our refactored code recommendation model.

References

[1] Aritra Ahmed, Ramiro Liscano, Akramul Azim, Yee-Kang Chan, and Vijay Sundaresan. 2024. Identification of Java lock contention anti-patterns based on run-time performance data. In *Proceedings of the 5th ACM/IEEE International Conference on Automation of Software Test (AST 2024)*. IEEE, 209–213.

[2] Aritra Ahmed, Ramiro Liscano, Akramul Azim, Yee-Kang Chang, and Vijay Sundaresan. 2024. Java lock contention anti-patterns and their associated recommendations. In *2024 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 144–151.

[3] Mohammad Mejbah Ul Alam, Tongping Liu, Guangming Zeng, and Abdullah Muzahid. 2017. Syncperf: Categorizing, detecting, and diagnosing synchronization performance bugs. In *Proceedings of the Twelfth European Conference on Computer Systems*. 298–313.

[4] James Bergstra and Yoshua Bengio. 2012. Random search for hyper-parameter optimization. *Journal of machine learning research* 13, 2 (2012), 505–509.

[5] Jialun Cao, Meiziniu Li, Ming Wen, and Shing-chi Cheung. 2023. A study on prompt design, advantages and limitations of chatgpt for deep learning program repair. *arXiv preprint arXiv:2304.08191* (2023).

[6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* 12 (2021), 12–18.

[7] Sidong Feng and Chunyang Chen. 2024. Prompting is all you need: Automated android bug replay with large language models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.

[8] Muhammad Usman Hadi, Qasem Al Tashi, Abbas Shah, Rizwan Qureshi, Amgad Muneer, Muhammad Irfan, Anas Zafar, Muhammad Bilal Shaikh, Naveed Akhtar, Jia Wu, et al. 2024. Large language models: a comprehensive survey of its applications, challenges, limitations, and future prospects. *Authorea Preprints* (2024), 10–16.

[9] Zeyu Han, Chao Gao, Jinyang Liu, Sai Qian Zhang, et al. 2024. Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608* (2024).

[10] Peter Hofer, David Gnedt, Andreas Schörgenhumer, and Hanspeter Mössenböck. 2016. Efficient tracing and versatile analysis of lock contention in Java applications on the virtual machine level. In *Proceedings of the 7th ACM/SPEC on International Conference on Performance Engineering*. 263–274.

[11] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021).

[12] IBM. 1999. Java Lock Monitor. http://perfinsp.sourceforge.net/examples.html#jlm

[13] Yiqiao Jin, Yunsheng Bai, Yanqiao Zhu, Yizhou Sun, and Wei Wang. 2023. Code recommendation for open source software developers. In *Proceedings of the ACM Web Conference 2023*. 1324–1333.

[14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.

[15] Zongxi Li, Xianming Li, Yuzhang Liu, Haoran Xie, Jing Li, Fu-lee Wang, Qing Li, and Xiaoqin Zhong. 2023. Label supervised llama finetuning. *arXiv preprint arXiv:2310.01208* (2023).

[16] Junyi Lu, Lei Yu, Xiaojia Li, Li Yang, and Chun Zuo. 2023. LLaMA-Reviewer: Advancing Code Review Automation with Large Language Models through Parameter-Efficient Fine-Tuning. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 647–658.

[17] Ehsan Mashhadi and Hadi Hemmati. 2021. Applying codebert for automated program repair of java simple bugs. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 505–509.

[18] Matt Post. 2018. A call for clarity in reporting BLEU scores. *arXiv preprint arXiv:1804.08771* 12 (2018), 12–18.

[19] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297* 12 (2020), 12–18.

[20] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.

[21] Ashadullah Shawon, Ramiro Liscano, Akramul Azim, Vijay Sundaresan, and Yee-Kang Chan. 2024. FaultLLAMA2: A Fine Tuned Large Language Model for Code Recommendations to Reduce Java Lock Contention. In *34th International Conference on Collaborative Advances in Software and Computing (CASCON)*. IEEE, 11–19.

[22] André Silva, Sen Fang, and Martin Monperrus. 2023. RepairLLaMA: Efficient Representations and Fine-Tuned Adapters for Program Repair. *arXiv preprint arXiv:2312.15698* (2023), 10–16.

[23] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023), 10–16.

[24] Kushala VM, Harikrishna Warrier, Yogesh Gupta, et al. 2024. Fine Tuning LLM for Enterprise: Practical Guidelines and Recommendations. *arXiv preprint arXiv:2404.10779* 18, 12 (2024), 12–18.

[25] Chunting Zhou, Pengfei Liu, Puxin Xu, Srinivasan Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al. 2024. Lima: Less is more for alignment. *Advances in Neural Information Processing Systems* 36 (2024), 10–16.